

第 8 章 CPU 的结构和功能

8.1 CPU 的结构

8.2 指令周期

8.3 中断系统

8.4 指令流水

本章从分析CPU功能及内部结构入手，讨论完成一条指令的全过程，以及为了提高数据处理能力、开发系统并行性的流水技术，进一步介绍中断相关技术



8.1 CPU 的结构

结构支撑功能，功能决定结构

一、CPU 的功能

本章重点介绍控制器功能

1. 控制器的功能

负责控制协调各部件，执行指令序列，基本功能：

取指令：形成指令地址、发出取指令的命令

指令控制

分析指令：分析操作内容和操作数有效地址

操作控制

执行指令：发出各种操作控制信号序列

控制运算器、存储器、I/O设备，完成指令

控制程序输入及结果的输出

时间控制

总线管理

处理异常情况和特殊请求

处理中断

2. 运算器的功能

已在前面章介绍

数据加工

实现算术运算和逻辑运算



二、CPU 结构框图

1. CPU 组成部分

指令控制

PC IR

操作控制

CU 时序电路

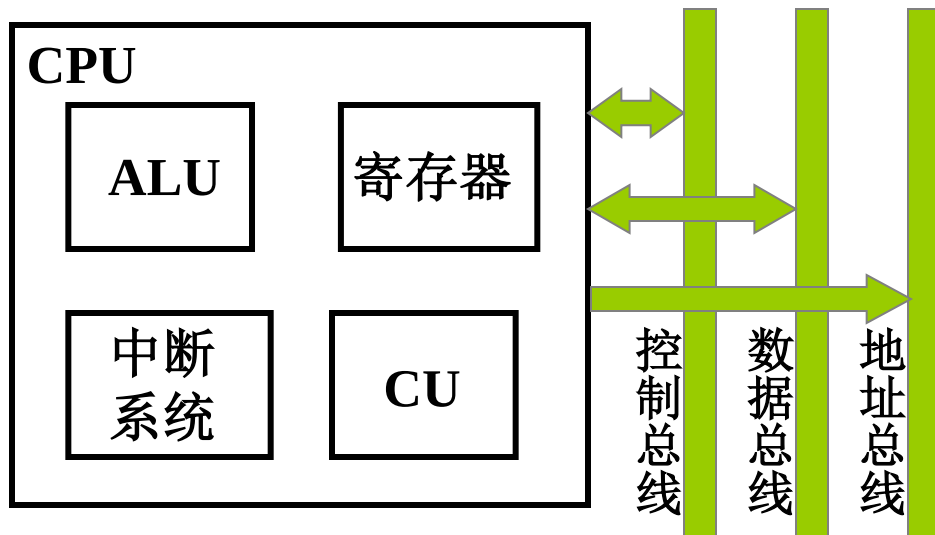
时间控制

数据加工

ALU 寄存器

处理中断

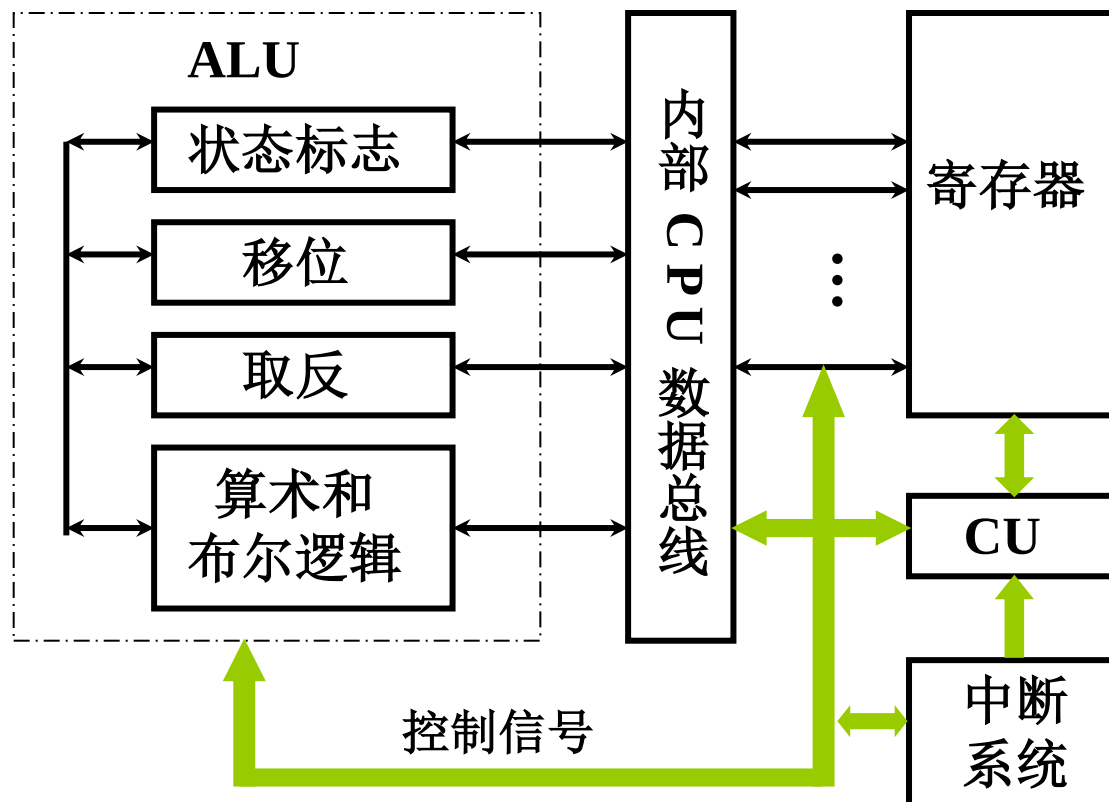
中断系统



图：CPU使用系统总线与其他部件传递信息



2. CPU 的内部结构



图：CPU 的内部结构——对上图的细化

三、CPU 的寄存器 在CPU内部、速度最快、位价最贵

1. 用户可见寄存器： 用户可编程改变其值，任务切换时需要保护现场

(1) 通用寄存器 存放操作数

在某些处理器里面，可以作为某种寻址方式的支撑，作为其专用寄存器

(2) 数据寄存器 存放操作数（满足各种数据类型）

两个寄存器拼接存放双倍字长数据

(3) 地址寄存器 存放地址，其位数应满足最大的地址范围

用于特殊寻址方式：段基址寄存器/堆栈指针

(4) 条件码寄存器 存放条件码，根据运算结果由硬件设置，如正、负、零、溢出、进位等，集中到一个寄存器中；可被编程测试，用作程序分支的依据；有些条件码可用指令设置



2. 控制和状态寄存器 被控制部件使用，用于控制CPU操作，有些对用户透明，用户不可对其编程

(1) 控制寄存器

例如寄存器：PC /MAR /MDR /IR , 取指令时：

PC → MAR → M → MDR → IR

其中 **MAR、MDR、IR** 用户不可见

PC 用户可见

(2) 状态寄存器

PSW 寄存器 存放程序状态字（条件码及其他信息）

中断标记寄存器

3. 举例 Z8000 8086 MC 68000的寄存器组织（**自主学习内容**）



四、控制单元 CU 和中断系统

1. CU 产生全部指令对应的微操作命令序列

组合逻辑设计

硬连线逻辑

参见后续章

微程序设计

存储逻辑

2. 中断系统

参见后续内容

五、ALU

参见第 6 章



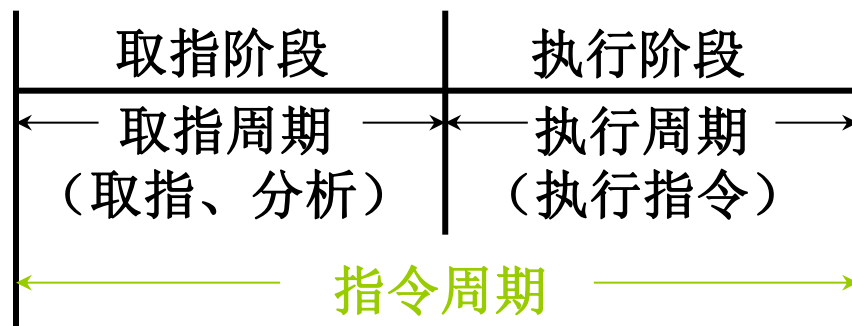
8.2 指令周期

一、指令周期的基本概念

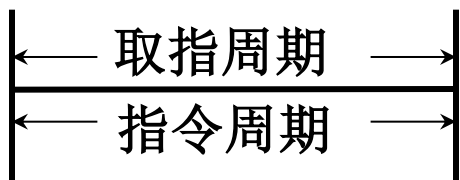
1. 指令周期

取出并执行一条指令所需的全部时间，指令周期可简单分为取指周期、执行周期两个阶段

完成一条指令 { 取指令、分析指令 取指周期
 执行指令 执行周期

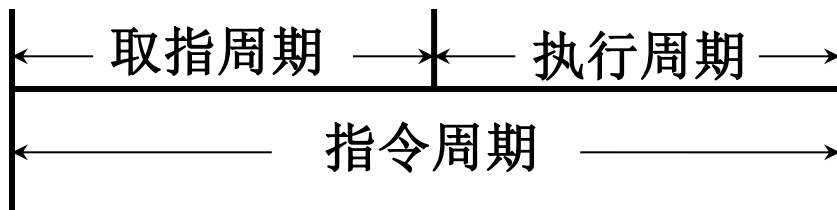


2. 每条指令的指令周期不同：指令操作复杂性不一样



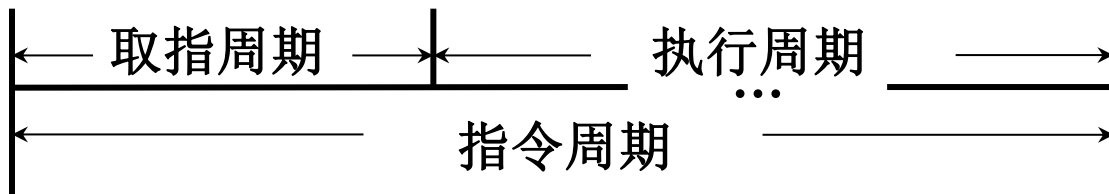
NOP

指令周期就是取指周期



ADD mem

指令周期包含两个存取周期



MUL mem

执行阶段操作复杂，
执行周期比加法指令更长



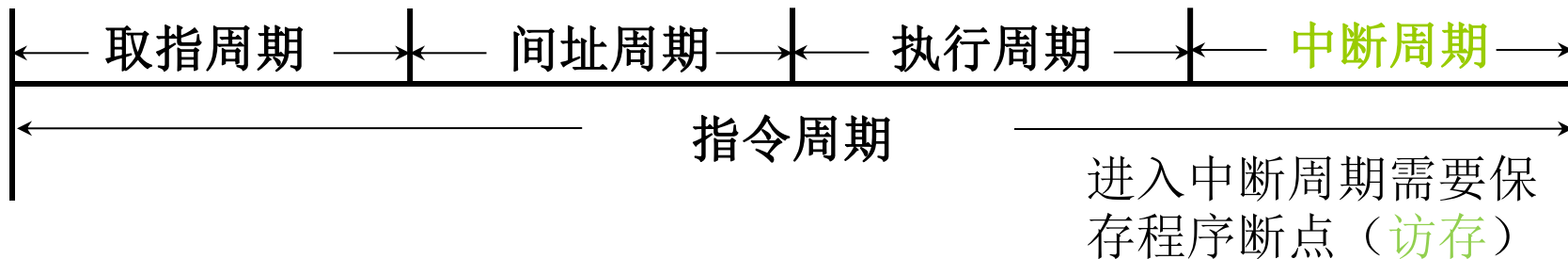
3. 具有间接寻址的指令周期

间址周期用于取操作数地址而访存



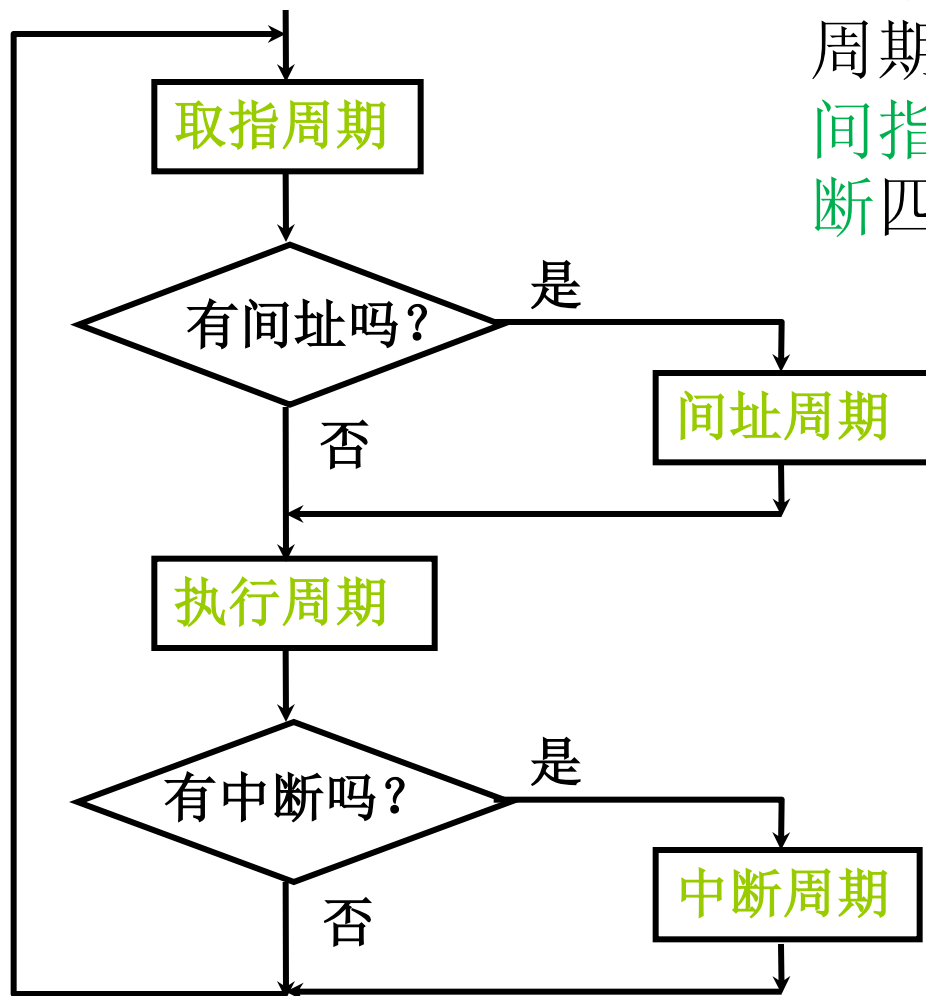
4. 带有中断周期的指令周期

CPU在每条指令执行阶段结束前，查询中断



5. 指令周期流程

一个完整的指令周期包含取指、间指、执行、中断四个子周期



6. CPU 工作周期的标志

每个子周期都有**CPU 访存**，共有四种作用：

取 指令

取指周期

取 有效地址

间址周期

取 操作数

执行周期

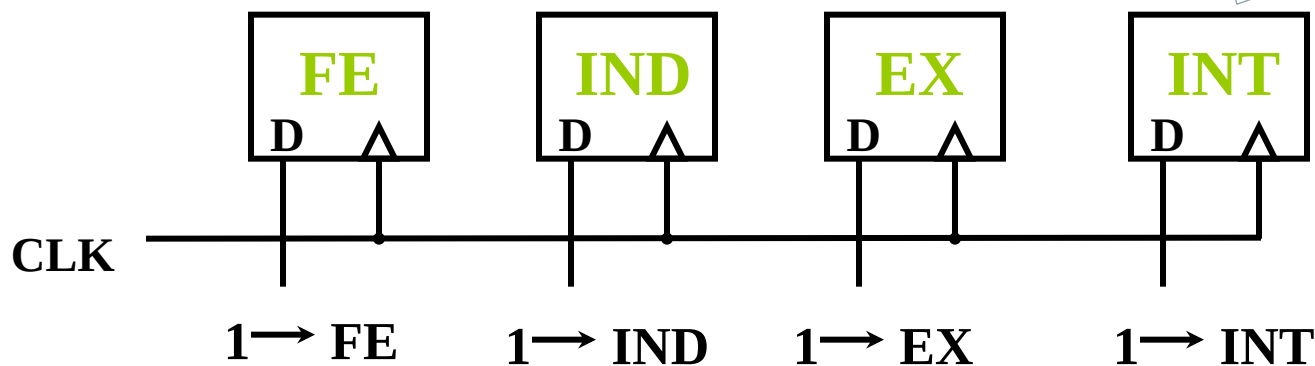
存 程序断点

中断周期

CPU 的

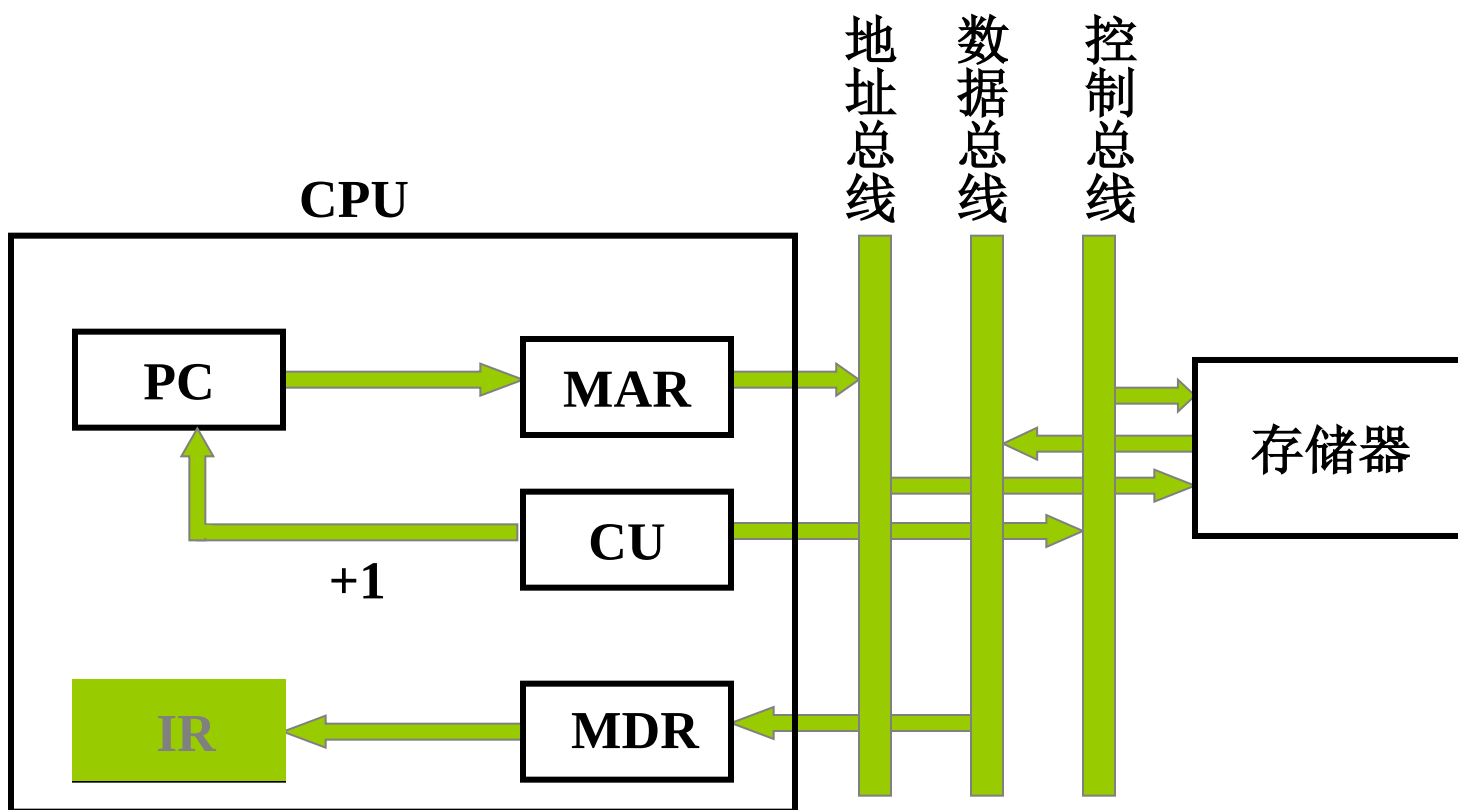
4个工作周期

各子周期标志触发器可用于控制该阶段的各个操作

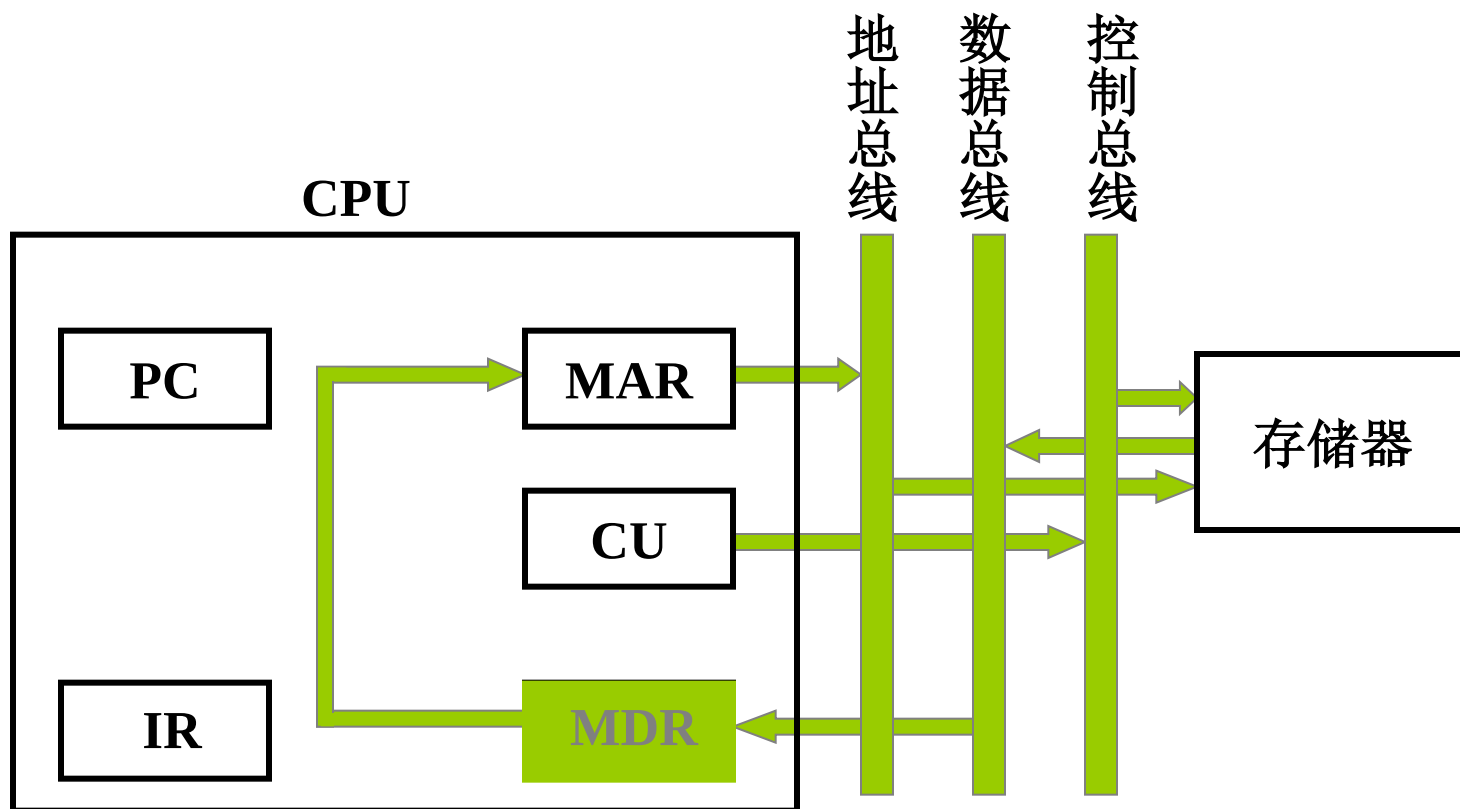


二、指令周期的数据流

1. 取指周期数据流



2. 间址周期数据流

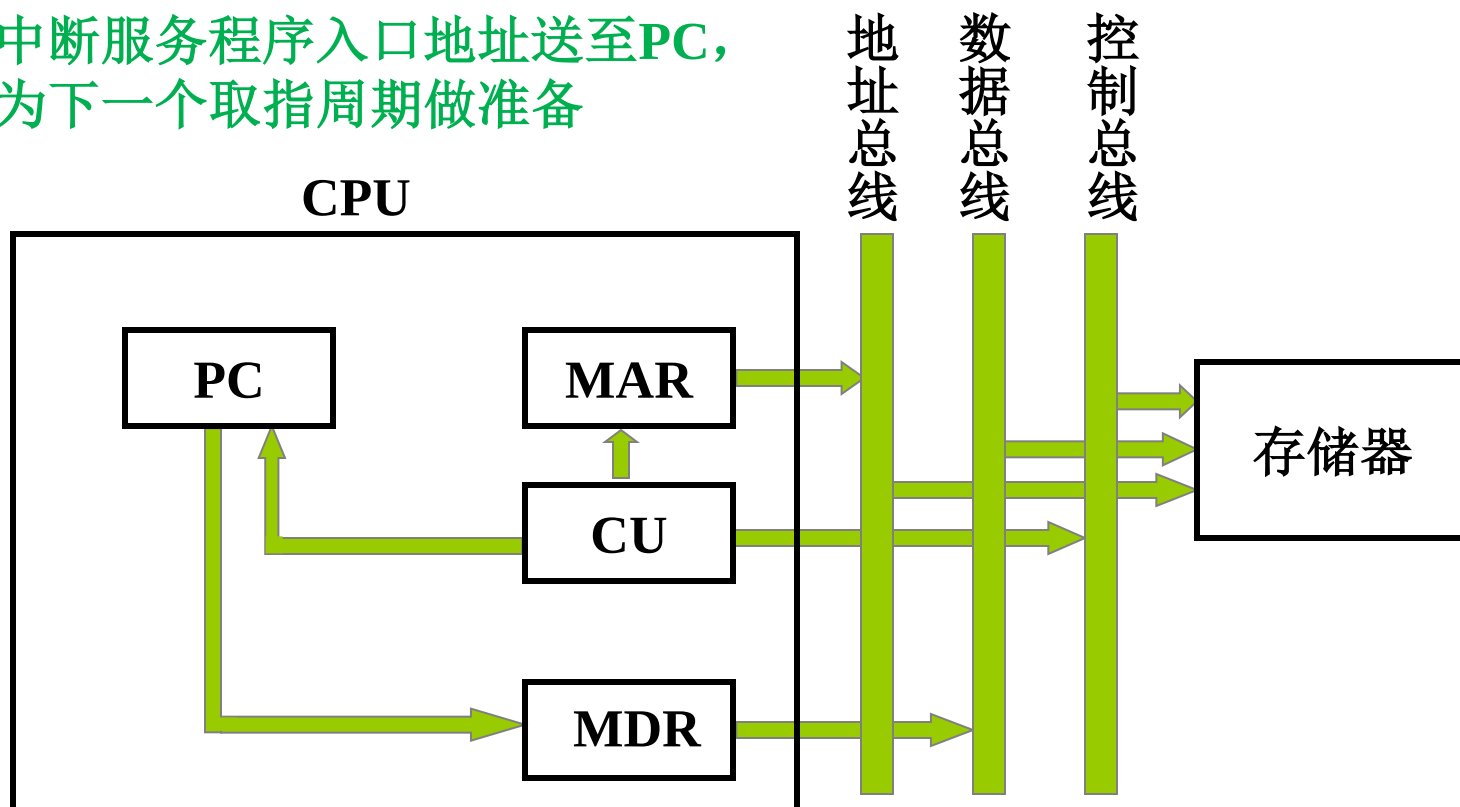


3. 执行周期数据流

不同指令的执行周期数据流不同

4. 中断周期数据流

- 保存PC当前内容，即断点地址
- 中断服务程序入口地址送至PC，为下一个取指周期做准备



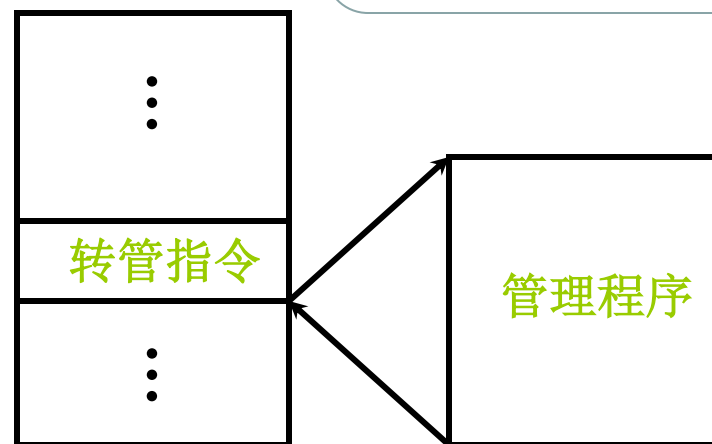
8.3 中断系统

一、概述

1. 引起中断的各种因素

(1) 人为设置的中断:

在程序中人为设置的中断，
如用于系统调用的软中断指令 `INT TYPE`



计算机系统存在多种中断，CPU中设有管理中断的机构

(2) 程序性事故：溢出、操作码不能识别、除法非法

(3) 硬件故障： 掉电、接触不良等

(4) I/O 设备： 外设准备就绪后发出中断请求

(5) 外部事件： 如实时控制系统中各种外部事件



2. 中断系统需解决的问题

- (1) 各中断源 如何 向 CPU 提出请求
- (2) 多个中断源 同时 提出 请求 怎么办
- (3) CPU 什么 条件、什么 时间、以什么 方式
响应中断
- (4) CPU 响应中断后，如何 保护现场
- (5) 如何停止原程序执行并转入中断程序入口地址
- (6) 中断处理结束后，如何 恢复现场，如何 返回
- (7) 处理中断的过程中又 出现新的中断 怎么办

方法：在中断系统中配置相应的硬件和软件



二、中断请求标记和中断判优逻辑 （解决前两个问题）

1. 中断请求标记 用于标记对应的中断源提出了中断请求
给每个中断源设置一个 中断请求标记触发器INTR
多个INTR 组成 中断请求标记寄存器

1	2	3	4	5			<i>n</i>
掉电	过热	主存读写校验错	阶上溢	非法除法		键盘输入	打印机输出

INTR可以 分散 在各个中断源的 接口电路中，或者：
INTR 集中在 CPU 的中断系统 内，组成寄存器



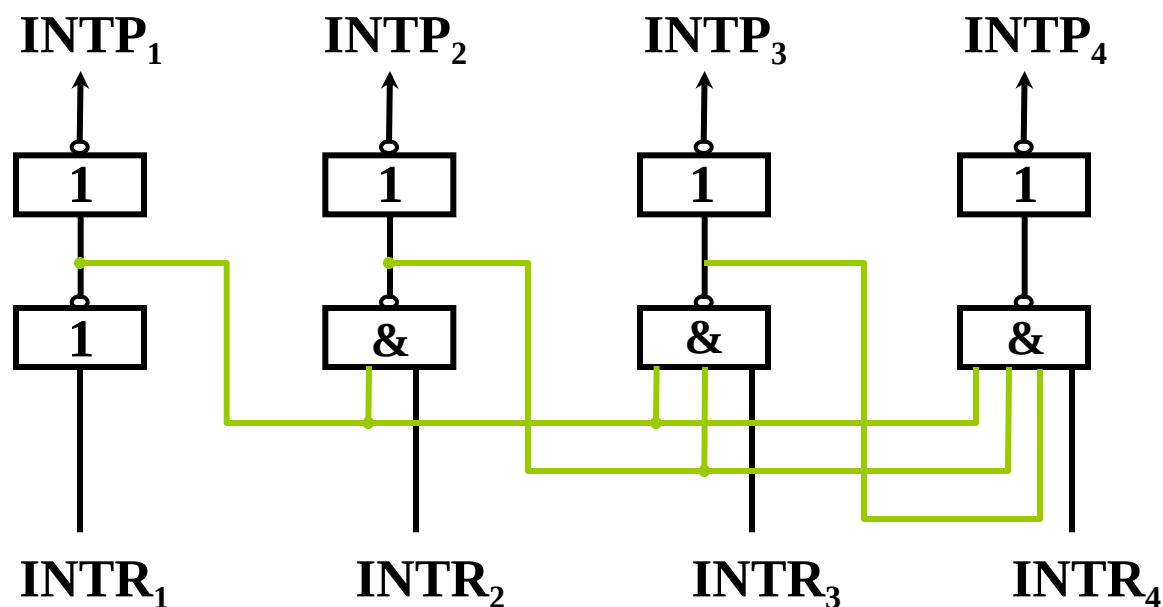
2. 中断判优逻辑 多个中断源同时请求时，按优先级次序予以响应

(1) 硬件实现（排队器）

① 分散 在各个中断源的 接口电路中 链式排队器

参见 第五章

② 集中在 CPU 内

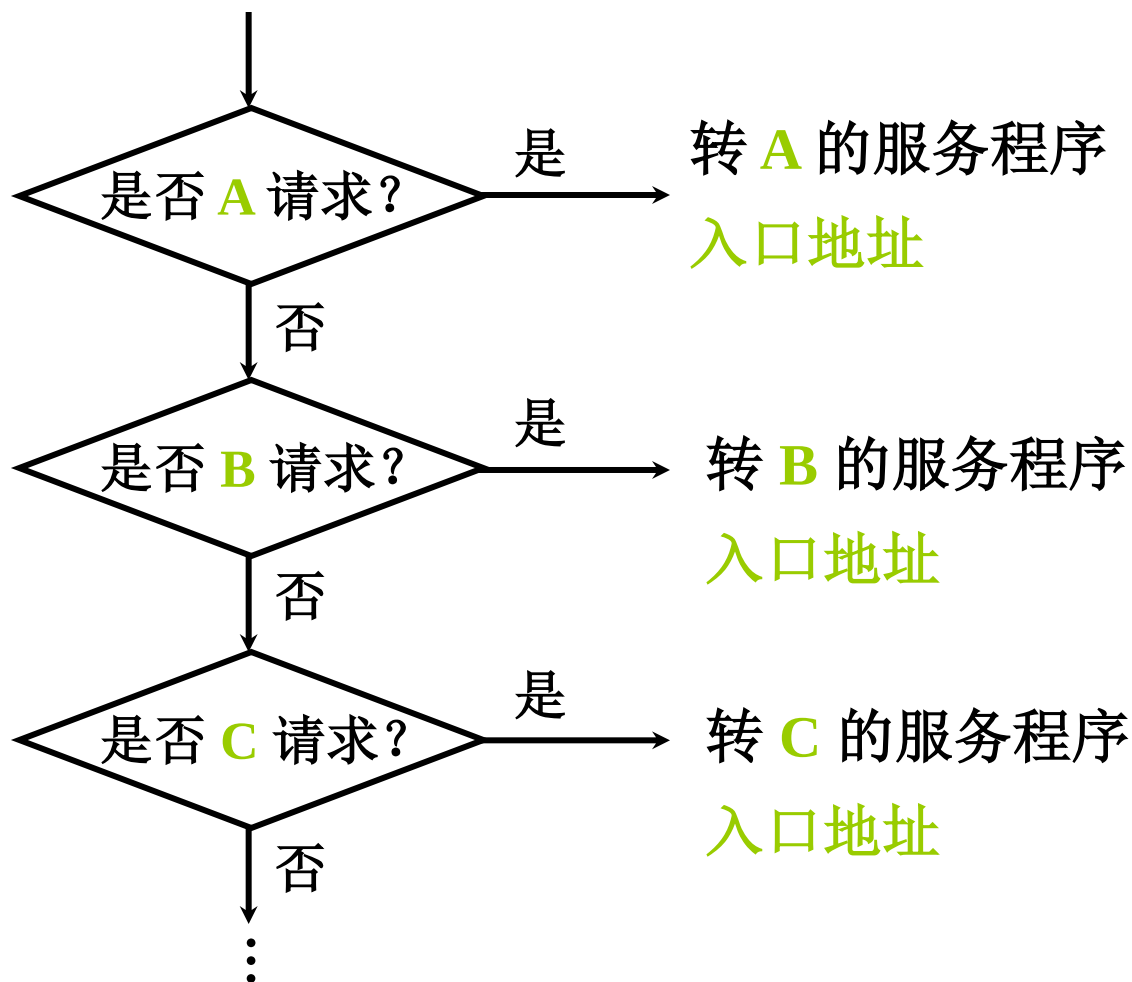


INTR₁、INTR₂、INTR₃、INTR₄ 优先级按降序排列



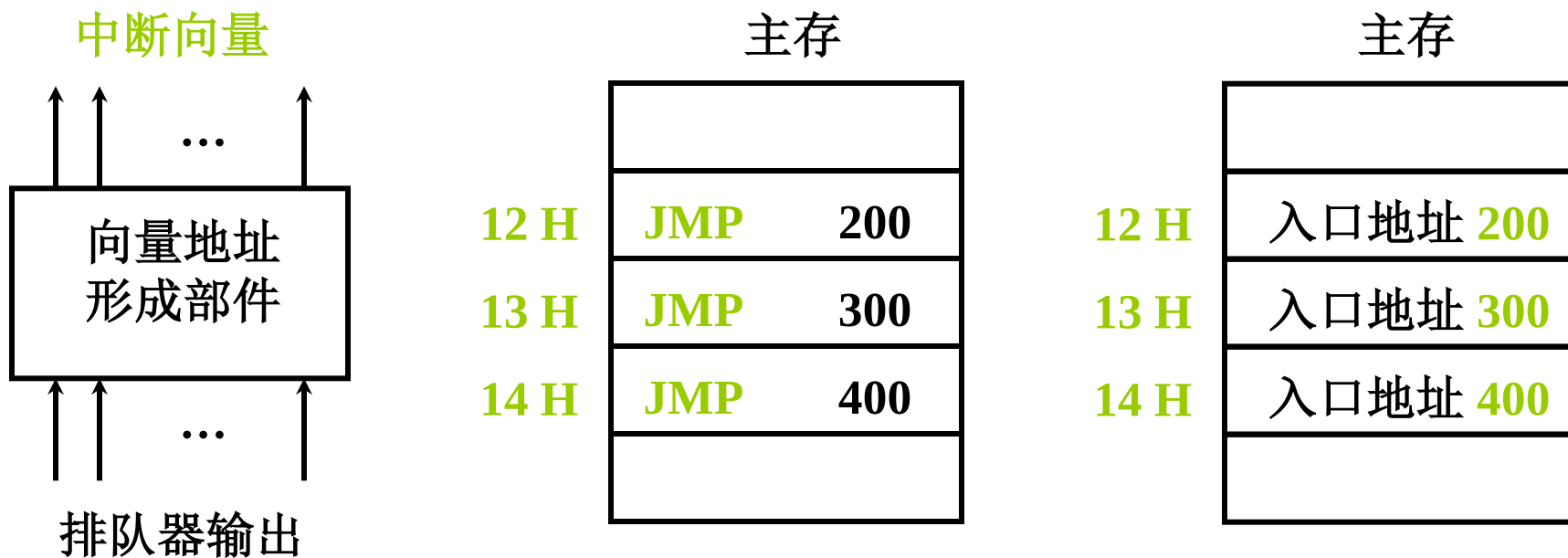
(2) 软件实现（程序查询）

A、B、C 优先级按 降序 排列
软件查询次序决定了优先级高低



三、中断服务程序入口地址的寻找

1. 硬件向量法 硬件产生向量地址，再由向量地址找到中断服务程序入口地址



中断向量地址 12H、13H、14H

ISR入口地址 200、300、400



2. 软件查询法 中断判优排队和寻找ISR入口地址同时完成

八个中断源 1, 2, ... 8 按 **降序** 排列

软件查询法速度
慢于硬件向量法

中断识别程序 (入口地址 **M**)

地 址	指 令	说 明
M	SKP DZ 1 [#]	1 [#] D = 0 跳 (D为完成触发器)
	JMP 1 [#] SR	1 [#] D = 1 转1 [#] 服务程序
	SKP DZ 2 [#]	2 [#] D = 0 跳
	JMP 2 [#] SR	2 [#] D = 1 转2 [#] 服务程序
	⋮	
	SKP DZ 8 [#]	8 [#] D = 0 跳
	JMP 8 [#] SR	8 [#] D = 1 转8 [#] 服务程序



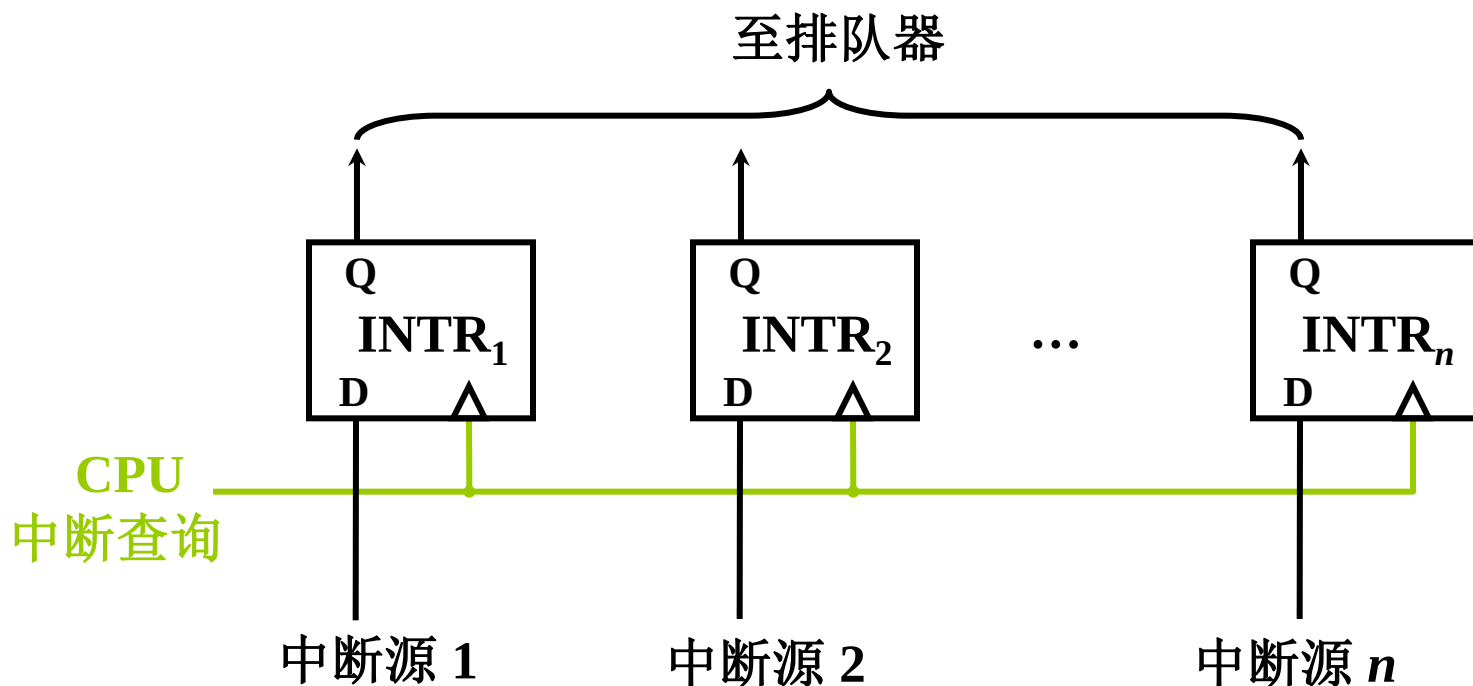
四、中断响应

1. 响应中断的条件

允许中断触发器 $EINT = 1$ ，且有中断请求

2. 响应中断的时间

指令执行周期结束时刻由CPU发查询信号



3. 中断隐指令

CPU响应中断后，在中断周期自动完成一系列操作，可认为是执行了一条隐指令，这些操作包括：

(1) 保护程序断点

断点存于存储器**特定单元**（如**0**地址），或者断点 **进栈**

(2) 寻找服务程序入口地址

向量地址 → PC（硬件向量法）

中断识别程序入口地址 $M \rightarrow PC$ (软件查询法)

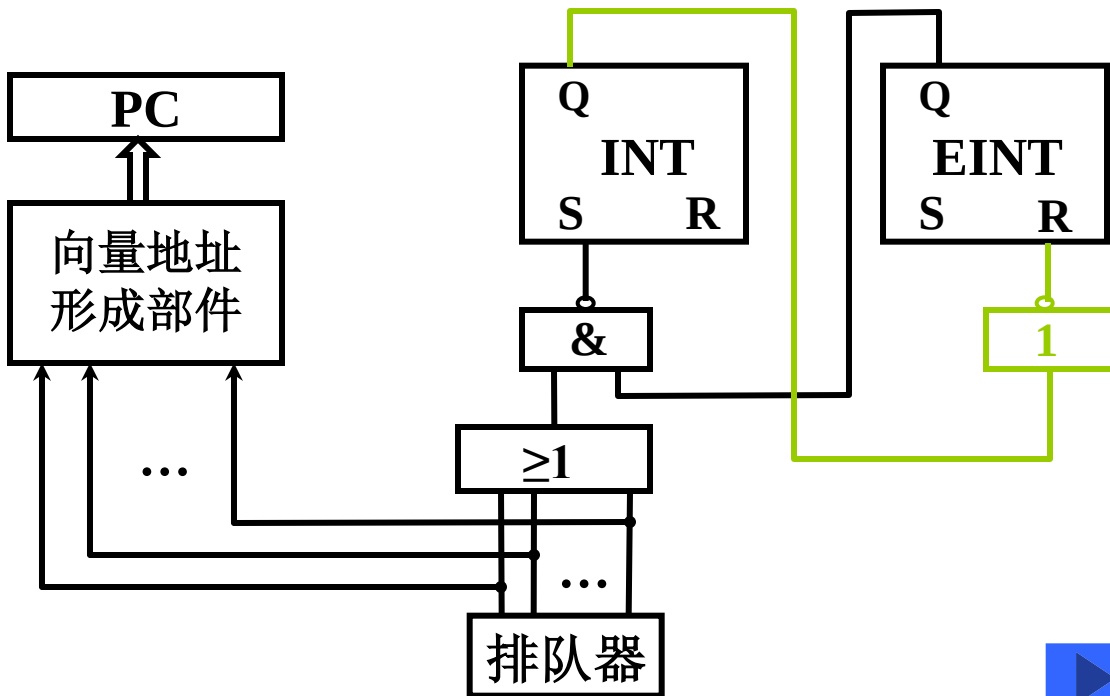
(3) 硬件关中断

中断响应期间禁止再次响应中断

INT 中断标记

EINT 允许中断

R-S 触发器



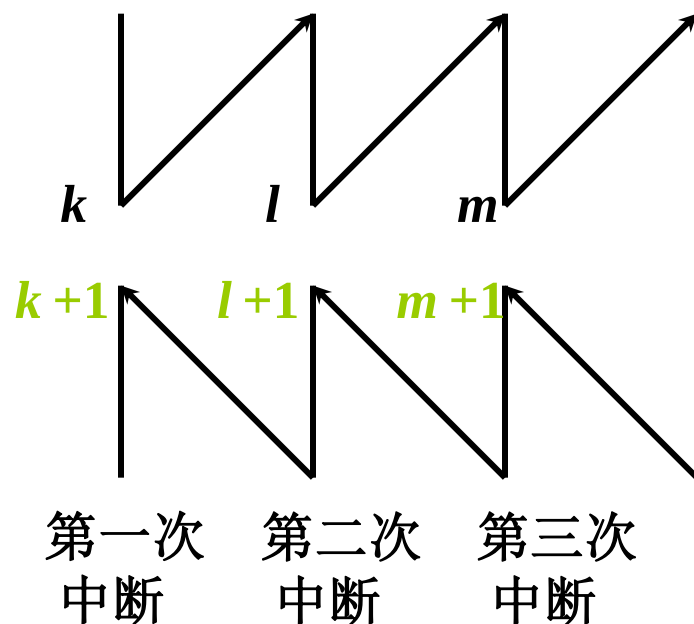
五、保护现场和恢复现场：为任务切换作准备

1. 保护现场
(现场即程序当前位置和运行状态)
 - 断点地址 中断隐指令 完成
 - 寄存器内容 中断服务程序 完成
2. 恢复现场 把寄存器内容恢复到中断处理前的状态，由中断服务程序 完成



六、中断屏蔽技术（用于多重中断）

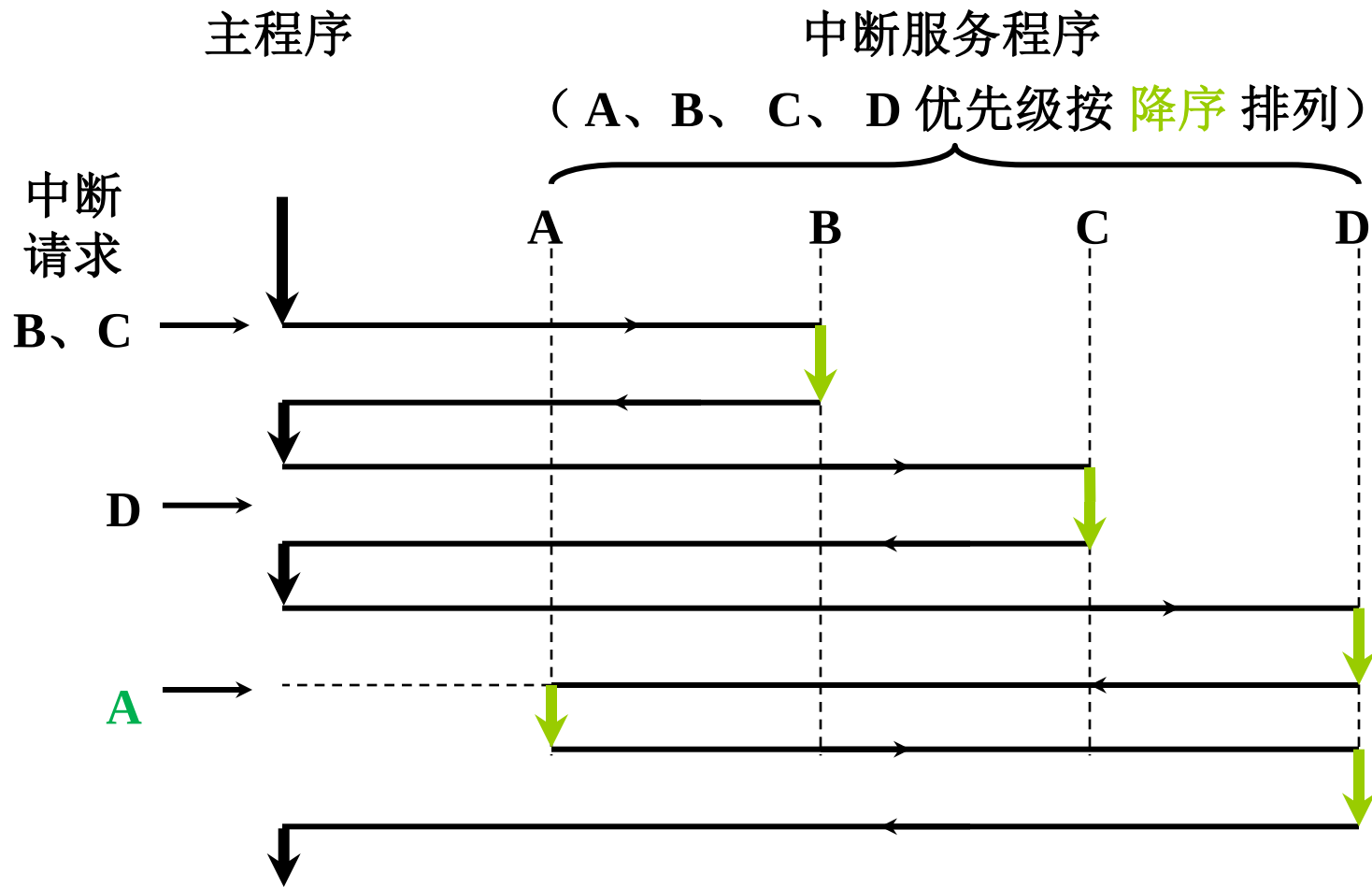
1. 多重中断的概念



程序断点 $k+1$, $l+1$, $m+1$

2. 实现多重中断的条件

- (1) 提前 设置 开中断 指令，在保护现场之后立即开中断
- (2) 优先级别高 的中断源 有权中断优先级别低 的中断源

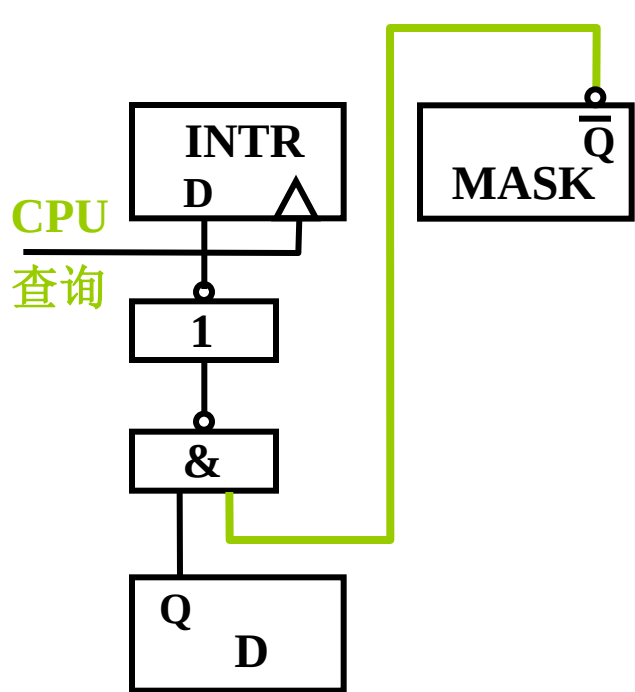


计算机中多重中断的实现机制

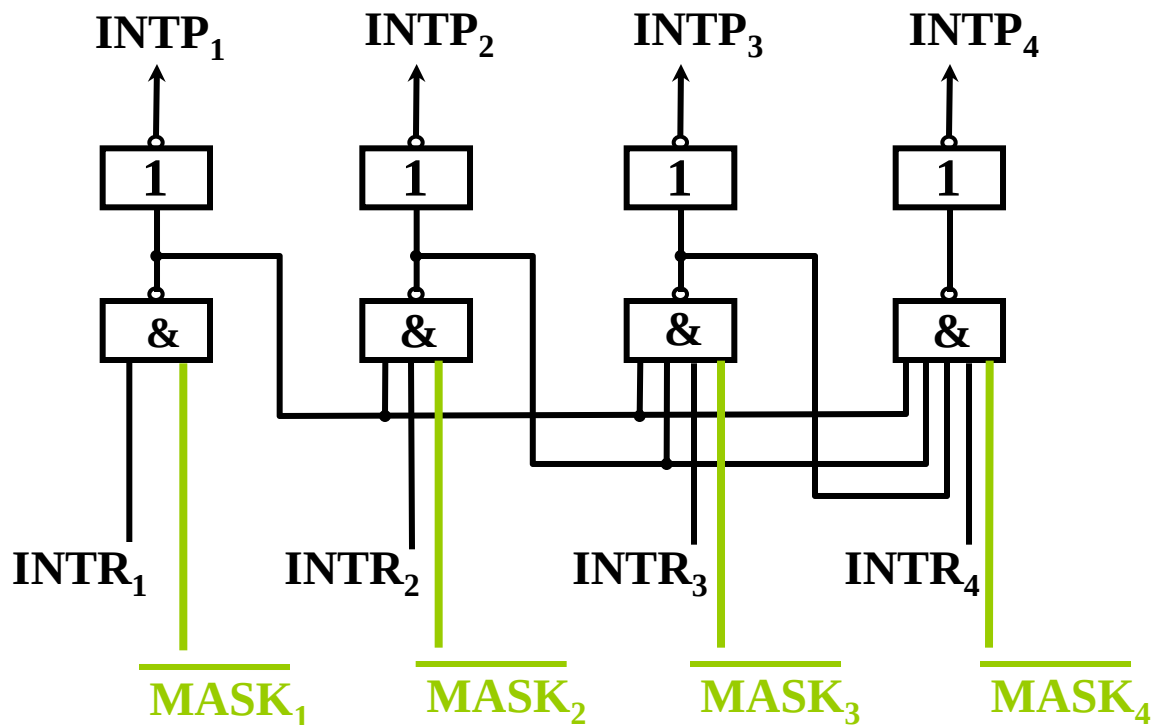
- 通过屏蔽技术（设置屏蔽字）调整由排队电路确定的优先级，使优先级安排更灵活；
- 主程序中屏蔽字为全零，第一层中断响应时，仅由排队电路确定中断优先级，称为响应优先级；
- 进入中断服务程序后，通过设置屏蔽字，改变对各中断源的优先级设定，开中断后，下一次中断时，比正在被响应的中断源优先级高的中断源才能参与排队，根据排队器和当前屏蔽字的共同作用，确定响应那个中断源；
- 每个中断服务程序中，都应设置自己的屏蔽字；
- 在中断服务程序中被调整的优先级（通过每个中断服务程序里的屏蔽字）起作用，确定了某个中断能否被优先完成处理，称处理优先级。

3. 屏蔽技术 保证低级别中断源不干扰高级别中断源的处理过程

(1) 屏蔽触发器的作用



a) 在中断源接口电路中
 $MASK = 0$ (未屏蔽)
 $INTR$ 能被置“1”



b) 在CPU中判优排队及屏蔽
 $MASK_i = 1$ (屏蔽)
 $INTP_i = 0$ (不能被排队选中)



(2) 屏蔽字

每个INTR对应一个屏蔽触发器，其全体组成一个屏蔽寄存器，屏蔽寄存器的内容称为屏蔽字，响应某个中断时，应该安排其对应的屏蔽字，以屏蔽某些中断源的请求

16个中断源 1, 2, 3, ..., 16 按 降序 排列

这里保留了与响应优先一样的优先级安排

优先级	屏蔽字															
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
3	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
4	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1
5	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
6	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1
⋮	屏蔽原则：只开放优先级比自己高的中断															
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1
16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1



(3) 屏蔽技术可改变“处理优先等级”

响应优先级 不可改变（排队器硬件决定）

处理优先级 可改变（在ISR中重新设置屏蔽字）

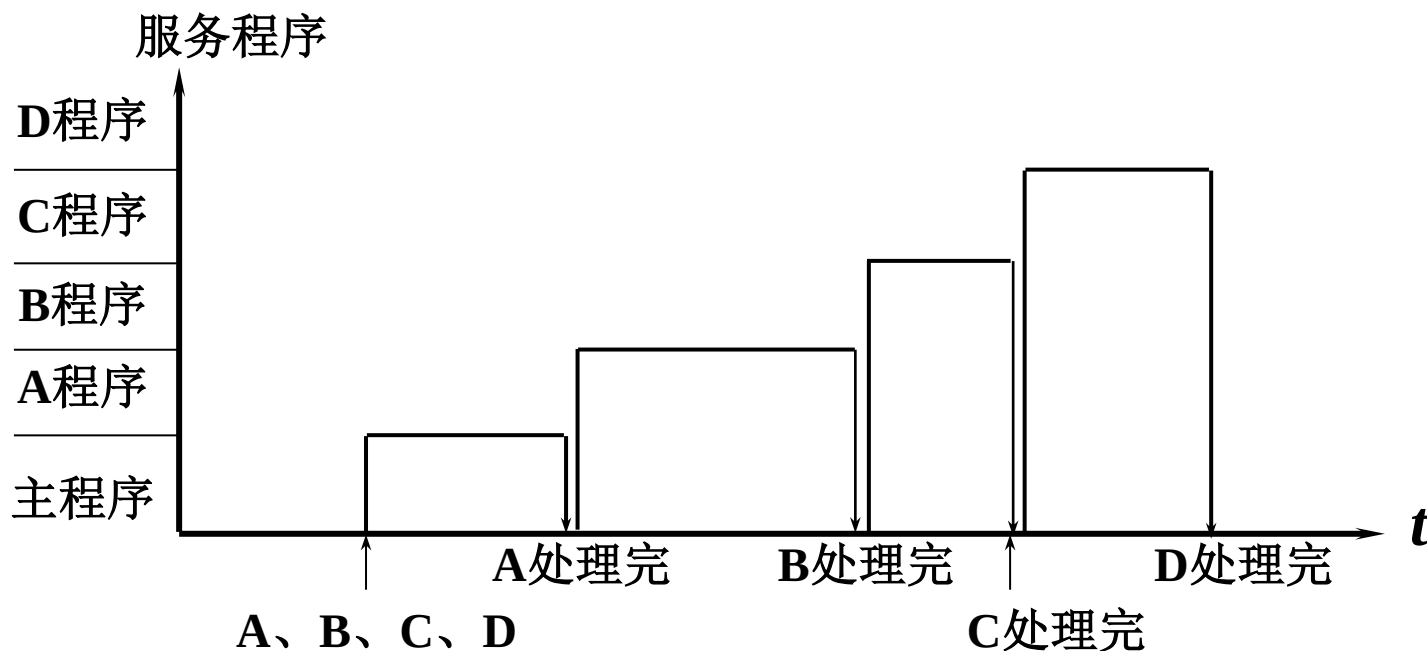
中断源	原屏蔽字	新屏蔽字
A	1 1 1 1	1 1 1 1
B	0 1 1 1	0 1 0 0
C	0 0 1 1	0 1 1 0
D	0 0 0 1	0 1 1 1

响应优先级 A→B→C→D 降序排列

处理优先级 A→D→C→B 降序排列



屏蔽技术改变处理优先等级

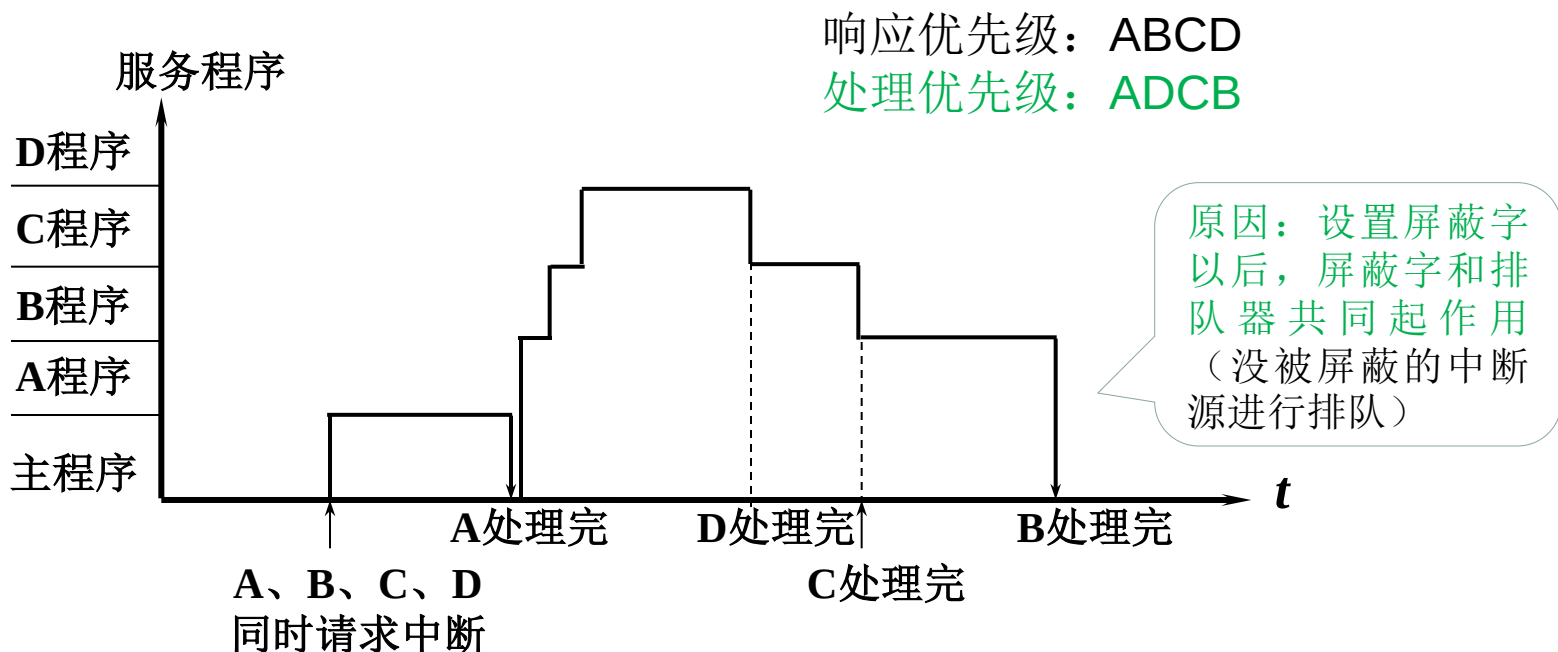


主程序中屏蔽字为全零不起作用

CPU 执行程序轨迹（原屏蔽字）



屏蔽技术改变处理优先等级



CPU 执行程序轨迹 (新屏蔽字)

(4) 屏蔽技术的其他作用

可以 人为地屏蔽 某个中断源的请求
便于程序控制



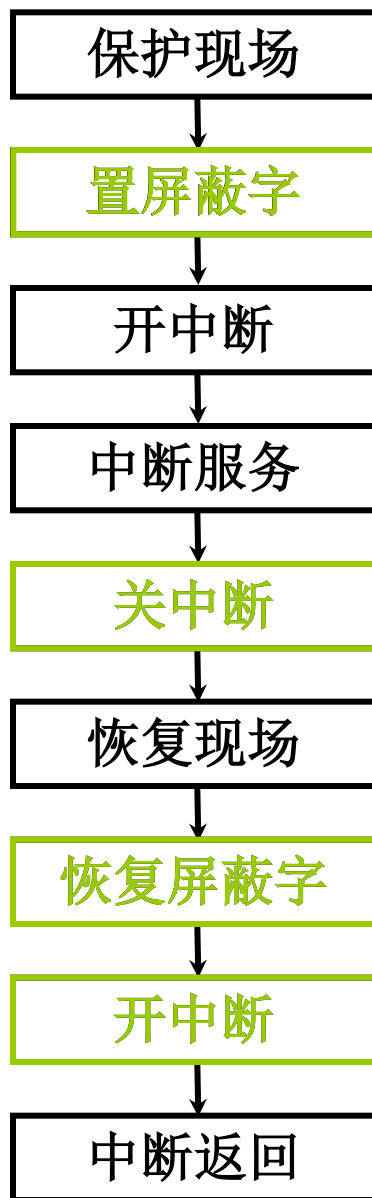
例题： 设某机有5个中断源L0, L1, L2, L3, L4
(按照中断响应的优先级次序从高到底排列),
现要将中断处理次序改为： L1, L4, L2, L0,
L3, 请写出各个中断源的屏蔽字

答：

中断源	位编号				
	0	1	2	3	4
L0	1	0	0	1	0
L1	1	1	1	1	1
L2	1	0	1	1	0
L3	0	0	0	1	0
L4	1	0	1	1	1

原则： 开放处理级别高于自己的中断--屏蔽位置0

(5) 新屏蔽字的设置



4. 多重中断的断点保护 不仅要保存，还可能需要保护

(1) 断点可以进栈 先进后出 中断隐指令 完成

(2) 断点也可以存入特定存储单元，如 “0” 地址单元 中断隐指令 完成

中断周期 0 $\rightarrow$ MAR

命令存储器写

PC $\rightarrow$ MDR 断点 $\rightarrow$ MDR

(MDR) $\rightarrow$ 存入存储器

若有三次中断，三个断点都存入 “0” 地址

如何保证断点不丢失？



(3) 程序断点存入“0”地址的断点保护

地 址	内 容	说 明
0	××××	存程序断点的内存单元
5	JMP SERVE	5 为中断向量向量地址
SERVE	STA SAVE	中断服务程序入口 保护现场
	⋮	
	LDA 0	} 0 地址内容转存
置屏蔽字	STA RETURN	
	→ ENI	开中断
	⋮	} 其他服务内容
	LDA SAVE	
	JMP @ RETURN	恢复现场
SAVE	××××	间址返回
RETURN	××××	存放 ACC 内容
		转存 0 地址内容



8.4 指令流水

一、如何提高机器速度

1. 提高访存速度

高速芯片

Cache

多体并行

2. 提高 I/O 和主机之间信息传送速度

中断

DMA

通道

I/O 处理机

多总线结构

3. 提高运算速度

高速芯片

改进算法

快速进位链

为了进一步提高整机处理能力：

- 提高器件性能--集成度已接近物理极限
- 改进系统结构--开发系统的并行性



二、系统的并行性

1. 并行的概念

并行 { **并发** 两个或两个以上事件在 **同一时间段** 发生
同时 两个或两个以上事件在 **同一时刻** 发生

- 能够并发或者同时完成两种以上功能，并且时间上互相重叠，就意味着存在并行性

2. 并行性的等级：并行性体现在不同的等级上

过程级（程序、进程） 粗粒度 软件实现

指令级（指令之间）	细粒度	硬件实现
（指令内部）		包括指令流水技术



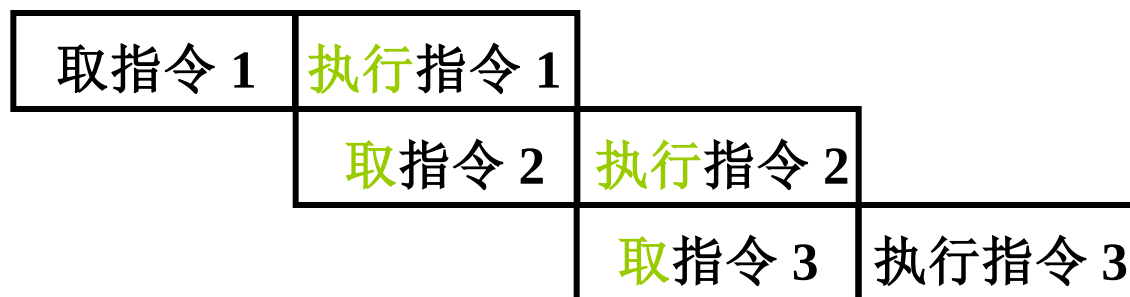
三、指令流水原理：假定指令处理简单分为两个阶段

1. 指令的串行执行：一段时间内只处理一条指令



取指令 取指令部件 完成 总有一个部件 空闲
执行指令 执行指令部件 完成

2. 指令的二级流水：两类部件同时处理两条指令



指令预取

若 取指 和 执行 阶段时间上 完全重叠

指令周期 减半 速度提高 1 倍



3. 影响指令流水效率加倍的因素

(1) 执行时间 > 取指时间



(2) 条件转移指令 对指令流水的影响

必须等 上条 指令执行结束，才能确定 下条 指令的地址，
造成时间损失 猜测法

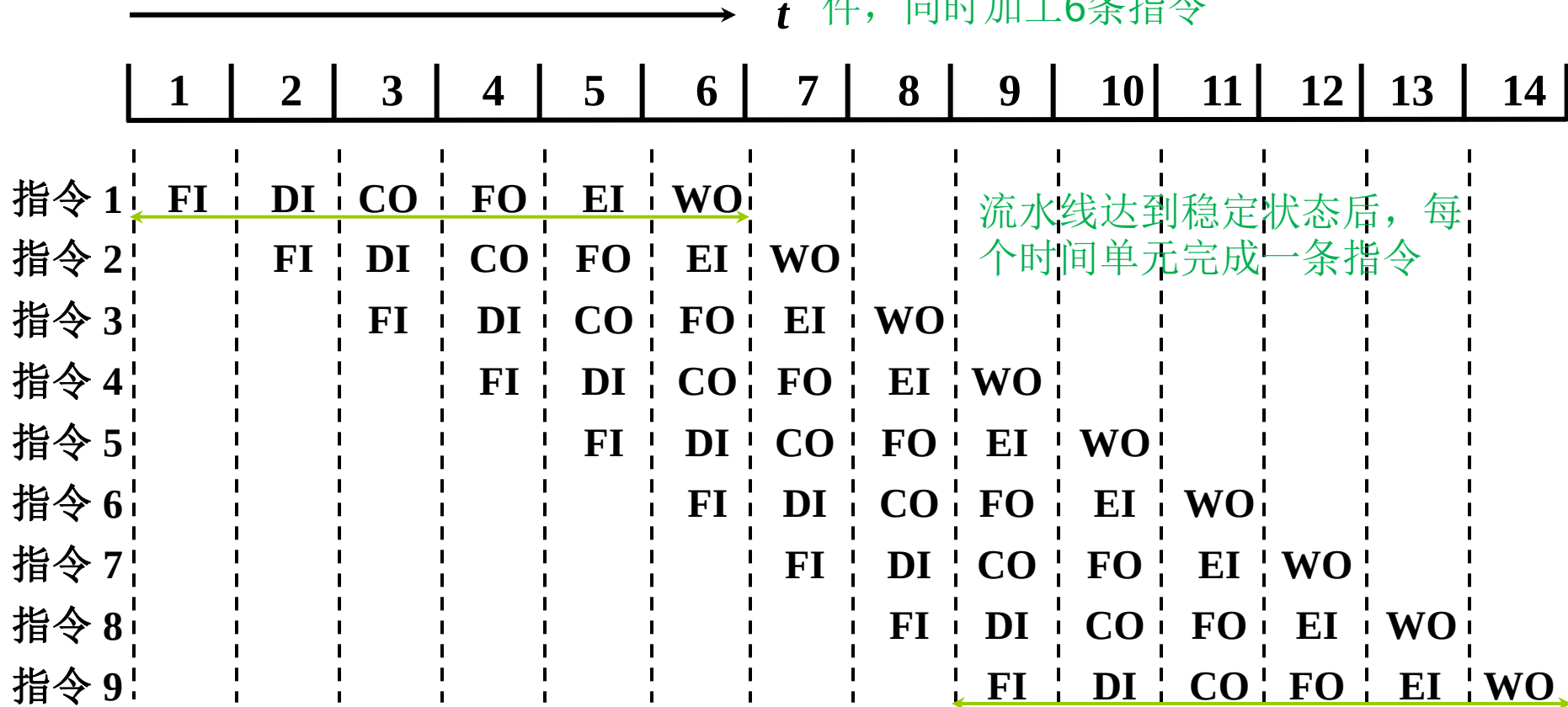
如何进一步提高处理速度？

把指令执行过程分解为更细的几个阶段，并行执行



4. 指令的六级流水

每个操作阶段完成需要一个时间单元，处理器有6个部件，同时加工6条指令



完成一条指令

串行执行

六级流水

6 个时间单元

$6 \times 9 = 54$ 个时间单元

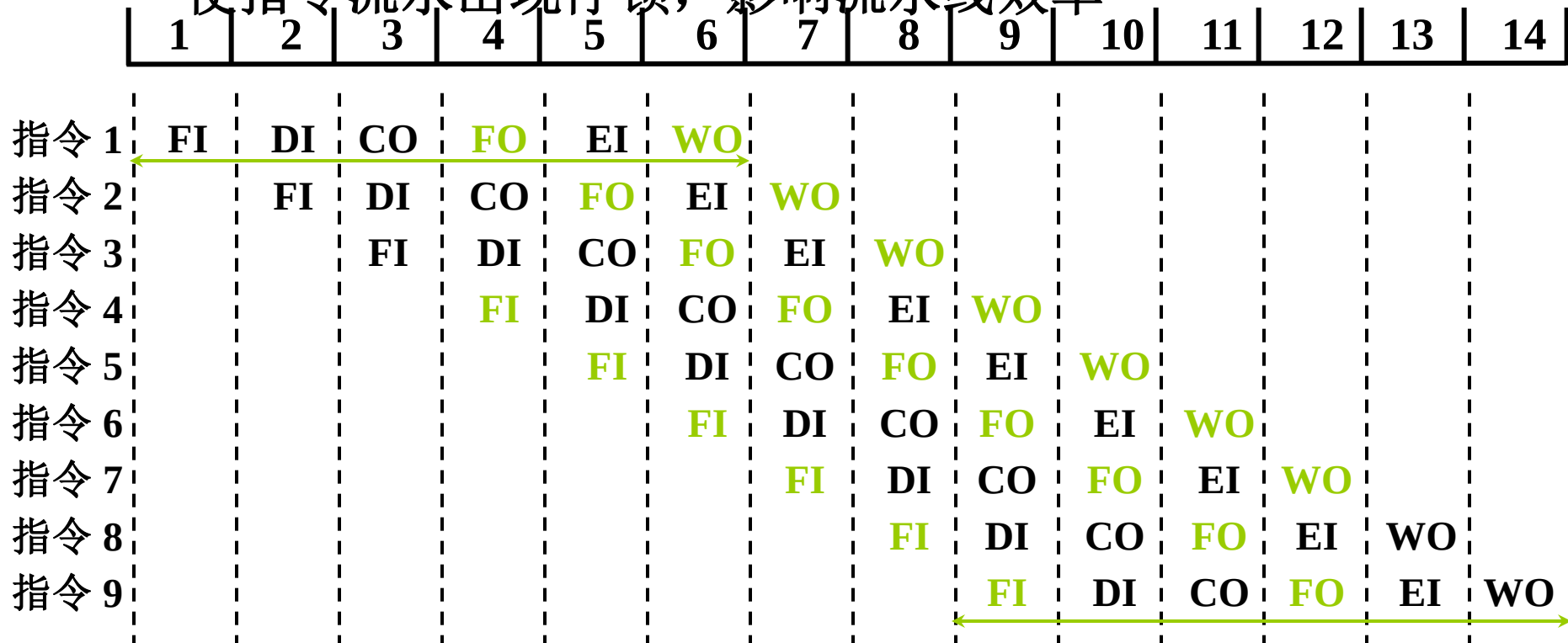
14 个时间单位



四、影响指令流水线性能的因素

1. 结构相关：不同指令共用同一功能部件产生资源冲突

使指令流水出现停顿，影响流水线效率



解决办法

- 停顿后一条指令操作
- 指令 1 与指令 4 冲突
- 指令 1、指令 3、指令 6 冲突
- 指令存储器和数据存储器分开
- 指令 2 与指令 5 冲突
- ...
- 指令预取技术（适用于访存周期短的情况）



2. 数据相关

不同指令因重叠操作，可能在操作数读/写（例如对寄存器操作）过程中改变正常的访问顺序（后继指令要用到前面指令执行结果）

- 写后读（正常顺序）相关（RAW）

SUB R_1, R_2, R_3 ; $(R_2) - (R_3) \rightarrow R_1$

ADD R_4, R_5, R_1 ; $(R_5) + (R_1) \rightarrow R_4$

- 读后写（正常顺序）相关（WAR）

STA M, R_2 ; $(R_2) \rightarrow M$ 存储单元

ADD R_2, R_4, R_5 ; $(R_4) + (R_5) \rightarrow R_2$

- 写后写（正常顺序）相关（WAW）

MUL R_3, R_2, R_1 ; $(R_2) \times (R_1) \rightarrow R_3$

SUB R_3, R_4, R_5 ; $(R_4) - (R_5) \rightarrow R_3$



数据相关解决方案：

- 后推法：相关后续指令延迟到所需操作数被写回到寄存器再执行
- 旁路技术：不等指令执行结果送回到寄存器之后，再取出，作为下一条指令的源操作数，而是直接把运算结果送至其他指令所需地方，这需要在ALU内部提供专门的旁路通道

3. 控制相关

由转移指令引起

→ M LDA # 0
 LDX # 0
 ADD X, D
 INX
 CPX # N
 BNE M
 DIV # N
 STA ANS

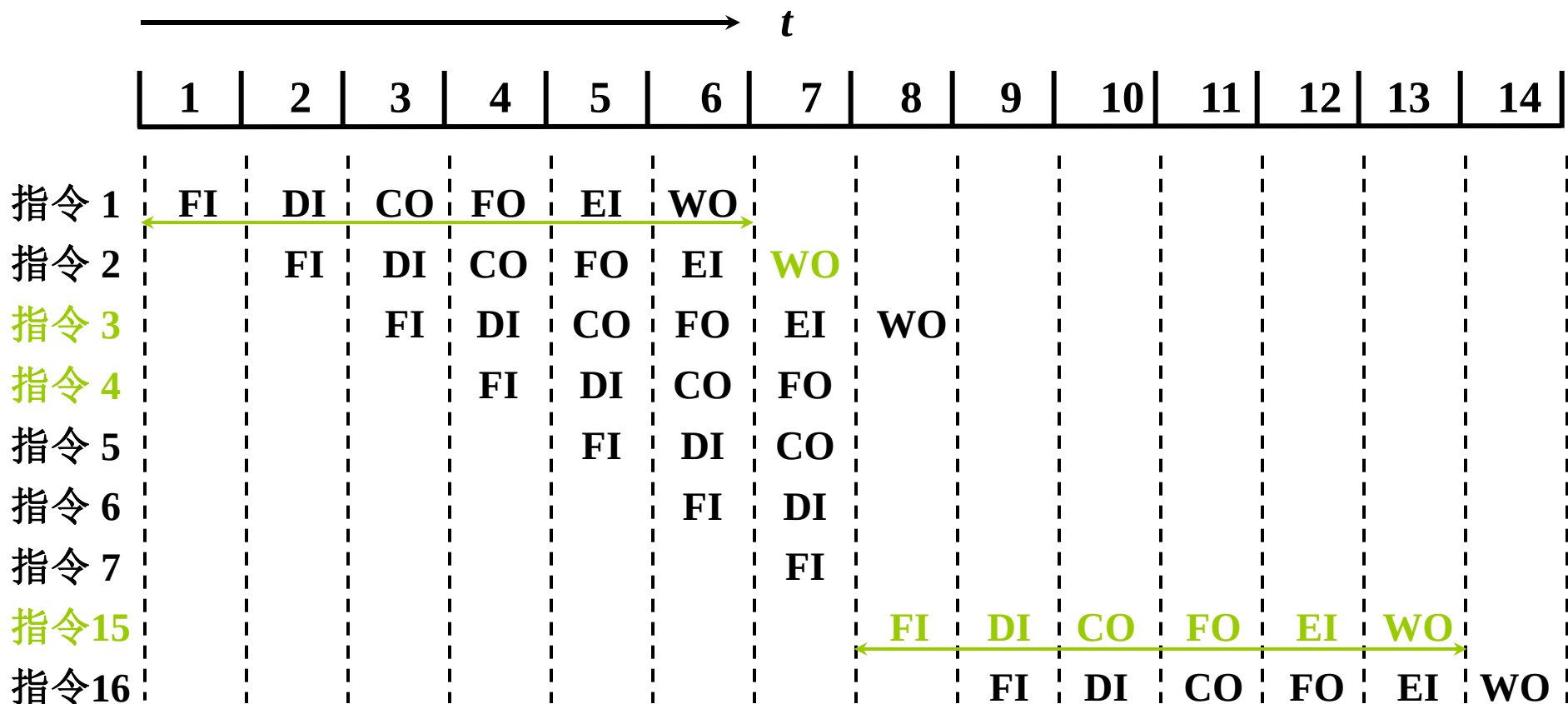
BNE 指令必须等
CPX 指令的结果
才能判断出
是转移
还是顺序执行



解决方案：尽早判别转移走向；预取不同程序走向上的目标指令；加快形成条件码等

控制相关对指令流水的影响

设 **指令3** 是条件转移指令，依赖指令2的执行结果



直到第7个时间单元才确定程序走向，
已取4-7条指令无效

转移损失
9~12没有指令完成



五、流水线性能

1. 吞吐率

单位时间内 流水线所完成指令 或 输出结果 的数量

设 m 段的流水线各段时间为 Δt

- 最大吞吐率：流水线连续流动达到稳定状态

$$T_{pmax} = \frac{1}{\Delta t}$$

- 实际吞吐率：完成 n 条指令的实际吞吐率

连续处理 n 条指令的吞吐率为

$$T_p = \frac{n}{m \cdot \Delta t + (n-1) \cdot \Delta t}$$



2. 加速比 S_p

m 段的流水线的速度与等功能的非流水线的速度之比

设流水线各段时间为 Δt

完成 n 条指令在 m 段流水线上共需

$$T = m \cdot \Delta t + (n-1) \cdot \Delta t$$

完成 n 条指令在等效的非流水线上共需

$$T' = nm \cdot \Delta t$$

则

$$S_p = \frac{nm \cdot \Delta t}{m \cdot \Delta t + (n-1) \cdot \Delta t} = \frac{nm}{m + n - 1}$$

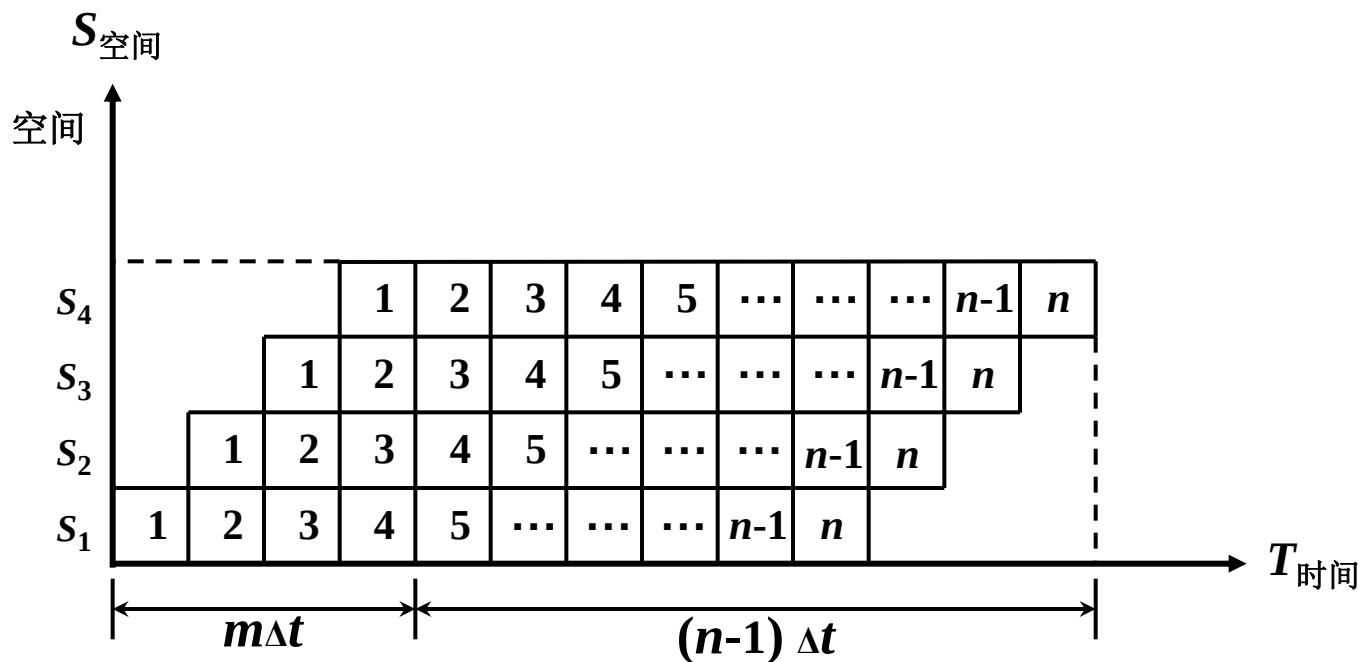


3. 效率

流水线中各功能段的 **利用率**

由于流水线有 **建立时间** 和 **排空时间**

因此各功能段的 **部件不可能** 一直 处于 **工作** 状态



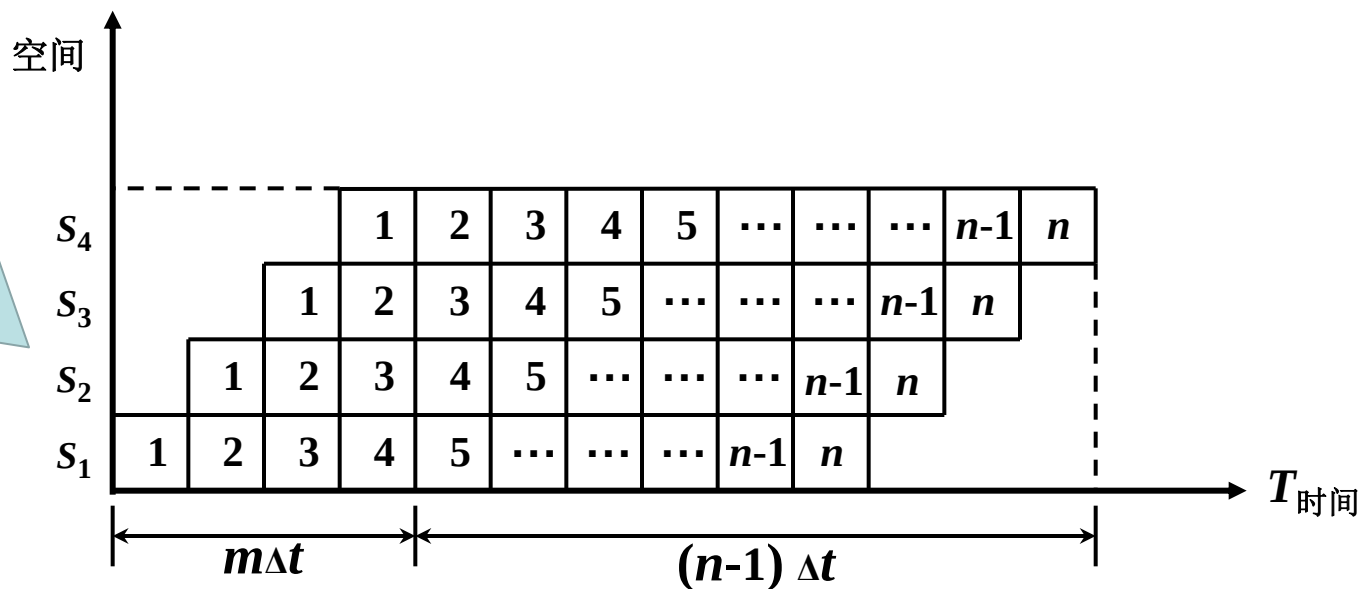
3. 效率

流水线中各功能段的 **利用率**

效率 = $\frac{\text{流水线各段处于工作时间的时空区}}{\text{流水线中各段总的时空区}}$

$$= \frac{mn\Delta t}{m(m+n-1)\Delta t}$$

纵轴
为流
水线
各段



例：假设指令流水线分取指、译码、执行、回写4个过程段，共有**10**条指令连续输入此流水线：

- (1) 假设时钟周期为**100ns**，求流水线的实际吞吐率
- (2) 求该流水线加速比

(1) 10条指令，4级流水线，完成全部指令需要的时间单元数： $4 + (10 - 1) = 13$

实际吞吐率 $10 / (100\text{ns} \times 13) = 0.77 \times 10^7$ 条指令/秒

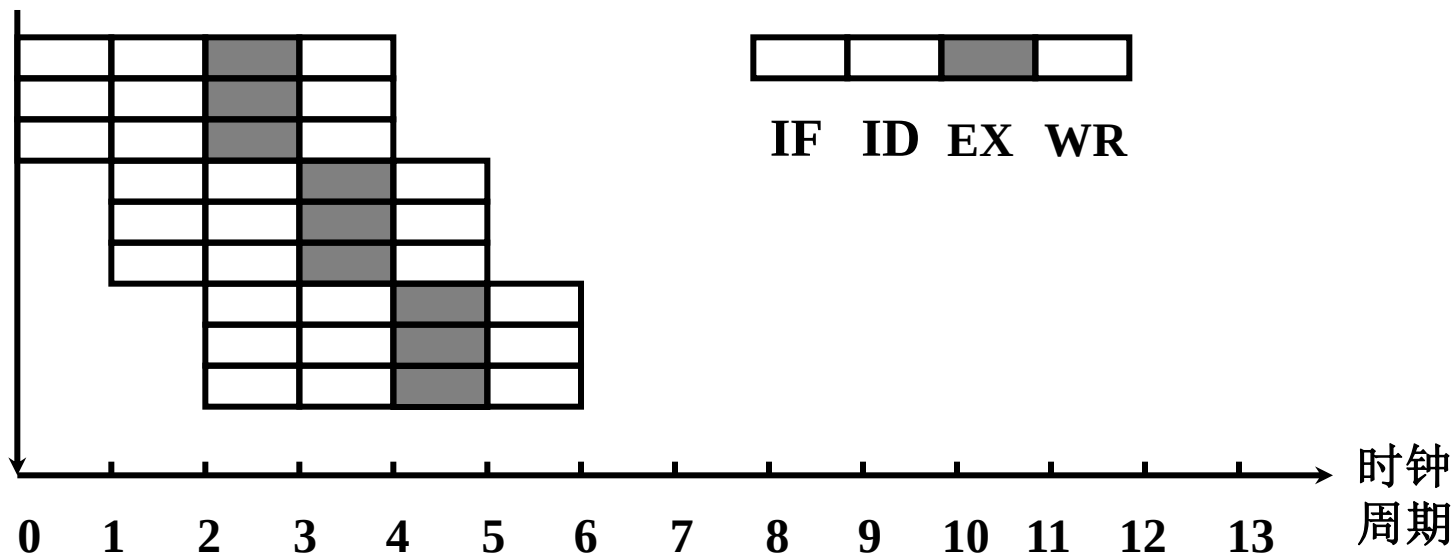
(2) 非流水线，10个指令需要 4×10 个时间单元；采用流水线，需要13个时间单元，因此加速比为： $40 \div 13 \approx 3$

六、流水线的多发技术

1. 超标量技术

- 每个时钟周期内可 **并发多条独立指令**：编译并执行需**配置多个功能部件**，以同时执行多个操作；
不存在数据相关的指令，才可以并行执行；
- 超标量计算机**不能重新安排**指令的**执行顺序**，而是通过编译优化技术，把可并行执行的指令搭配起来

指令序列



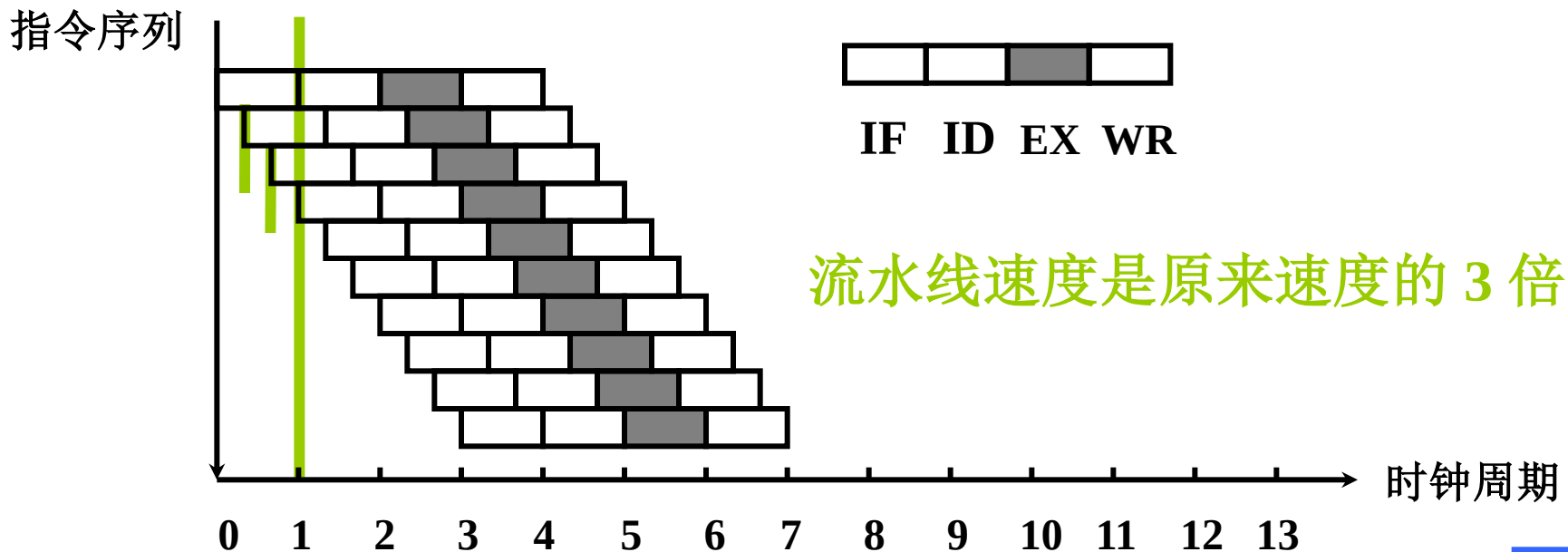
2. 超流水线技术

- 在一个流水线段内再分段（3段）

在一个时钟周期内 一个功能部件使用多次（3次），
流水线速度成为原来的多倍

- 不能调整 指令的 执行顺序

靠编译程序解决优化问题



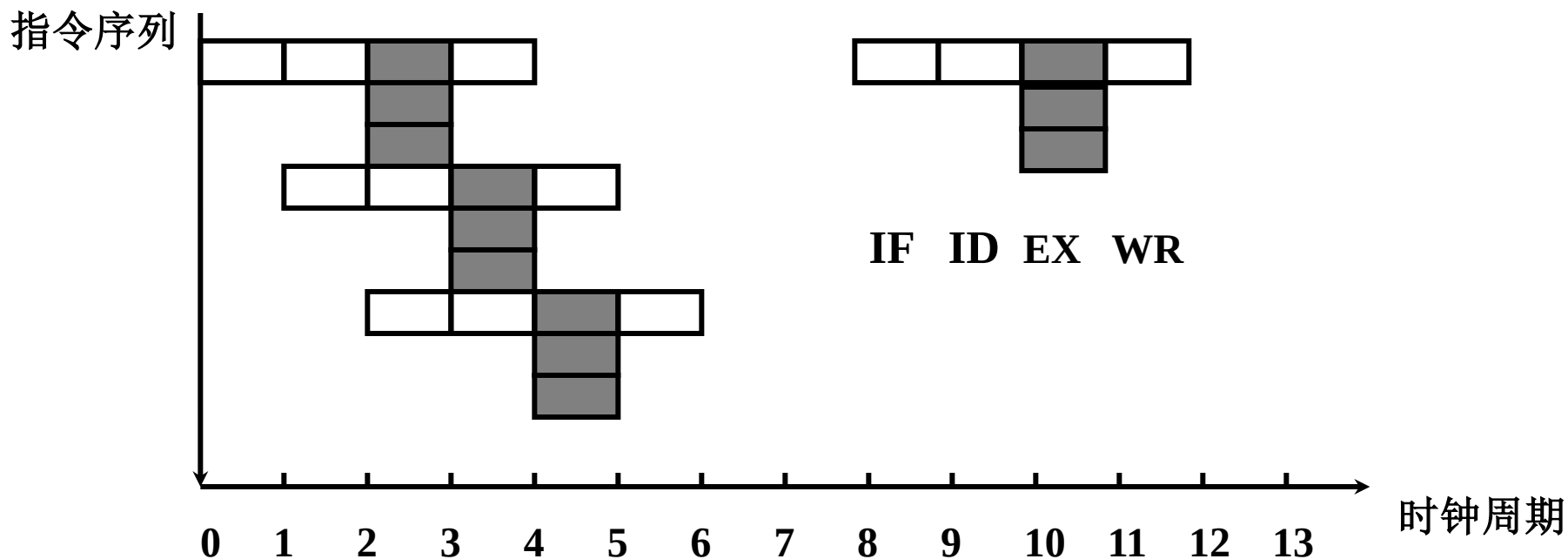
3. 超长指令字技术 多条指令在多个功能部件并行处理

➤ 由编译程序 挖掘 出指令间 潜在的 并行性

将 多条 能 并行操作 的指令组合成 一条具有

多个操作码字段的 超长指令字（可达几百位）

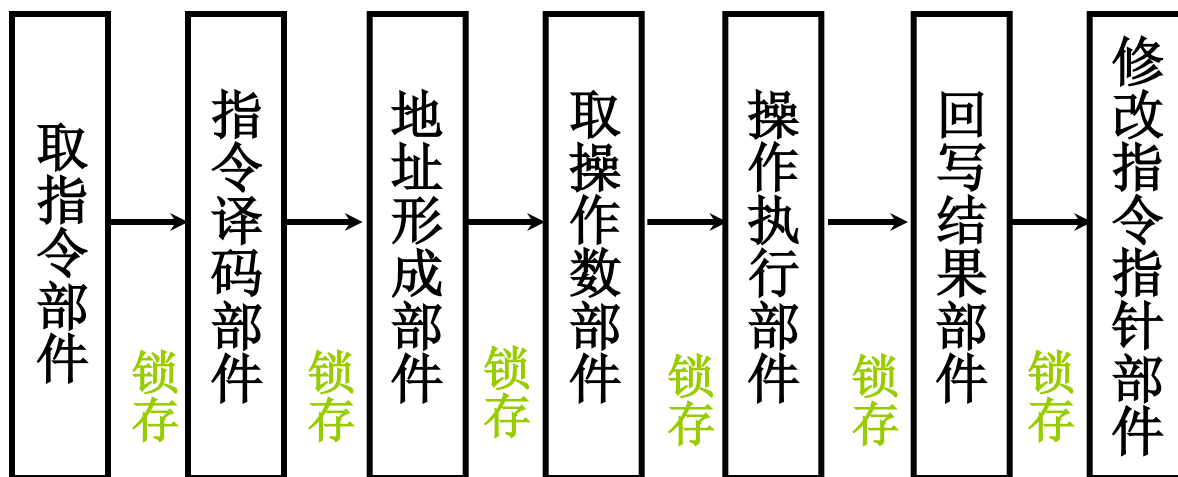
➤ 每个操作码控制一个功能部件，同时执行多条指令



七、流水线结构

1. 指令流水线结构：指令级流水线

完成一条指令分 7 段，每段需一个时钟周期



若 流水线不出现断流 1 个时钟周期出 1 结果

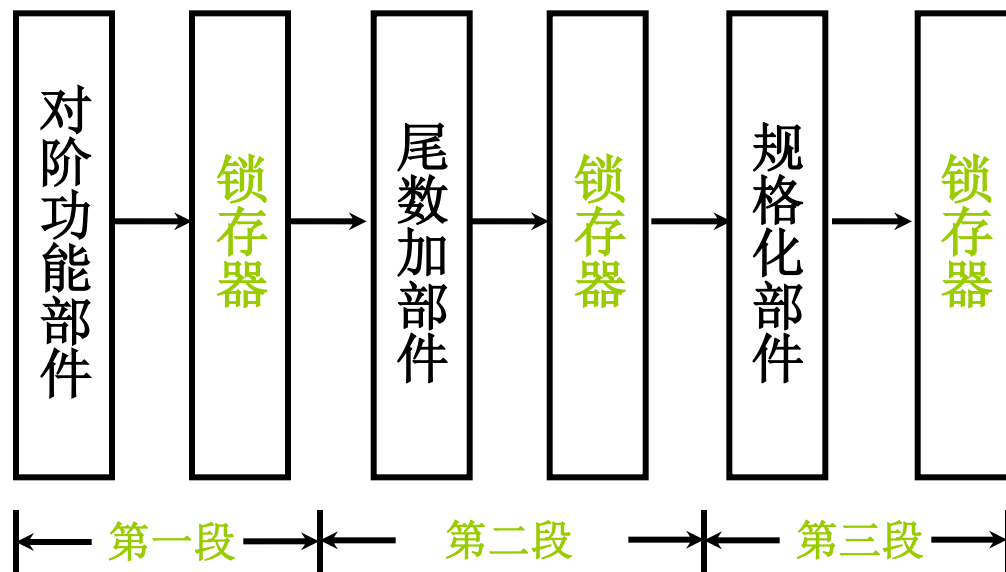
不采用流水技术 7 个时钟周期出 1 结果

理想情况下，7 级流水 的速度是不采用流水技术的 7 倍



2. 运算流水线：部件级的流水线技术

例如，完成 浮点加减 运算 可分
对阶、尾数求和、规格化 三段



分段原则：每段 操作时间尽量 一致，需合理安排；
流水线相邻两段执行不同操作，两段之间需要设置
锁存器，保证一个时钟周期内输入信号不变



作业

- 教材第371-372页：
第5、7、10、12、17、18、19、22、25，
26题

Thank you